

HDOC Day-32 — Matrix Operations Using Separate Functions

Name- Aaryan Pratap Singh

Sap id - 590032842

Batch- 15

Algorithm

Main Algorithm

1. Start.
2. Display the menu.
3. Read the user's choice.
4. Use `switch`:
 - Choice 1 → call `additionCommutative()`.
 - Choice 2 → call `multiplicationNonCommutative()`.
 - Choice 3 → exit.
 - Otherwise → display "Invalid choice!".
5. Repeat until choice 3.
6. Stop.

Function 1: Matrix Addition

1. Read rows and columns.
2. Validate that both are positive.
3. Declare two VLA matrices `A` and `B`.
4. Read both matrices.
5. Calculate `A + B`.
6. Calculate `B + A`.
7. Display both results.
8. Compare corresponding elements.
9. If all are equal, display that matrix addition is commutative.

Function 2: Matrix Multiplication

For `AB` and `BA` to both be defined using the same two matrices, choose:

- `A` of order $m \times n$
- `B` of order $n \times m$

Then:

- `AB` has order $m \times m$
- `BA` has order $n \times n$

1. Read `m` and `n`.

2. Declare VLA matrices $A[m][n]$ and $B[n][m]$.
3. Read their elements.
4. Calculate AB .
5. Calculate BA .
6. Display both products.
7. If the products have different dimensions, they cannot be equal.
8. Otherwise, compare their corresponding elements.
9. Report whether the particular matrices commute.

C program

```
#include <stdio.h>

void inputMatrix(int rows, int cols, int a[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

void displayMatrix(int rows, int cols, int a[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

void additionCommutative()
{

```

```

int rows, cols, i, j, equal = 1;

printf("\nEnter number of rows and columns: ");
scanf("%d %d", &rows, &cols);

while (rows <= 0 || cols <= 0)
{
    printf("Rows and columns must be positive.\n");
    printf("Enter again: ");
    scanf("%d %d", &rows, &cols);
}

int A[rows][cols];
int B[rows][cols];
int AB[rows][cols];
int BA[rows][cols];

printf("\nEnter elements of matrix A:\n");
inputMatrix(rows, cols, A);

printf("\nEnter elements of matrix B:\n");
inputMatrix(rows, cols, B);

/* A + B and B + A */
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        AB[i][j] = A[i][j] + B[i][j];
        BA[i][j] = B[i][j] + A[i][j];
    }
}

printf("\nA + B:\n");
displayMatrix(rows, cols, AB);

printf("\nB + A:\n");
displayMatrix(rows, cols, BA);

/* Compare A+B and B+A */
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        if (AB[i][j] != BA[i][j])

```

```

        {
            equal = 0;
        }
    }
}

if (equal)
    printf("\nA + B = B + A\n"
           "Hence, matrix addition is commutative.\n");
else
    printf("\nA + B != B + A\n");
}

void multiplicationNonCommutative()
{
    int m, n, i, j, k, equal = 1;

    printf("\nMatrix A will be of order m x n.\n");
    printf("Matrix B will be of order n x m.\n");

    printf("Enter m and n: ");
    scanf("%d %d", &m, &n);

    while (m <= 0 || n <= 0)
    {
        printf("Dimensions must be positive.\n");
        printf("Enter m and n again: ");
        scanf("%d %d", &m, &n);
    }

    int A[m][n];
    int B[n][m];
    int AB[m][m];
    int BA[n][n];

    printf("\nEnter elements of matrix A (%d x %d):\n", m,
n);
    inputMatrix(m, n, A);

    printf("\nEnter elements of matrix B (%d x %d):\n", n,
m);
    inputMatrix(n, m, B);

    /* Calculate AB */
    for (i = 0; i < m; i++)

```

```

{
    for (j = 0; j < m; j++)
    {
        AB[i][j] = 0;

        for (k = 0; k < n; k++)
        {
            AB[i][j] += A[i][k] * B[k][j];
        }
    }
}

/* Calculate BA */
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        BA[i][j] = 0;

        for (k = 0; k < m; k++)
        {
            BA[i][j] += B[i][k] * A[k][j];
        }
    }
}

printf("\nA x B:\n");
displayMatrix(m, m, AB);

printf("\nB x A:\n");
displayMatrix(n, n, BA);

/*
    If m != n, AB and BA have different orders,
    so they cannot be equal.
*/
if (m != n)
{
    printf("\nAB and BA have different orders.\n");
    printf("Therefore, AB != BA.\n");
    printf("This demonstrates that matrix multiplication
"
           "is not commutative in general.\n");
    return;
}

```

```

/* If dimensions are same, compare elements */
for (i = 0; i < m; i++)
{
    for (j = 0; j < m; j++)
    {
        if (AB[i][j] != BA[i][j])
        {
            equal = 0;
        }
    }
}

if (equal)
{
    printf("\nFor these particular matrices, AB = BA.
\n");
    printf("These matrices commute, although matrix "
        "multiplication is not commutative in general.
\n");
}
else
{
    printf("\nAB != BA\n");
    printf("Hence, this example demonstrates that matrix
"
        "multiplication is not commutative.\n");
}
}

int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Prove matrix addition is commutative\n");
        printf("2. Demonstrate matrix multiplication is not
commutative\n");
        printf("3. Exit\n");
        printf("=====\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
}

```

```

switch (choice)
{
    case 1:
        additionCommutative();
        break;

    case 2:
        multiplicationNonCommutative();
        break;

    case 3:
        printf("\nExiting program...\n");
        break;

    default:
        printf("\nInvalid choice! Enter 1, 2, or
3.\n");
}

} while (choice != 3);

return 0;
}

```

Test Case Table

Test	Operation	Input	Expected Result
1	Addition	$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$A+B = B+A$
2	Addition	2×3 matrices	Both sums identical
3	Multiplication	$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$AB \neq BA$
4	Multiplication	A is 2×3 , B is 3×2	AB is 2×2 and BA is 3×3 , so unequal
5	Validation	$m = 0, n = 2$	Ask for dimensions again
6	Menu	Choice 7	Invalid choice!
7	Exit	Choice 3	Program terminates

Execution — Option 1

```
[aaryanrajput@aaryans-macbook ~ % nano day32.c
[aaryanrajput@aaryans-macbook ~ % clang day32.c -o day32.c
[aaryanrajput@aaryans-macbook ~ % ./day32.c

===== MENU =====
1. Prove matrix addition is commutative
2. Demonstrate matrix multiplication is not commutative
3. Exit
=====
Enter your choice: 1

Enter number of rows and columns: 2 2

Enter elements of matrix A:
1 2 3 4

Enter elements of matrix B:
5 6 7 8

A + B:
6      8
10     12

B + A:
6      8
10     12

A + B = B + A
Hence, matrix addition is commutative.
```

Execution — Option 2

```
===== MENU =====
1. Prove matrix addition is commutative
2. Demonstrate matrix multiplication is not commutative
3. Exit
=====
Enter your choice: 2

Matrix A will be of order m x n.
Matrix B will be of order n x m.
Enter m and n: 2 2

Enter elements of matrix A (2 x 2):
1 2 3 4

Enter elements of matrix B (2 x 2):
5 6 7 8

A x B:
19      22
43      50

B x A:
23      34
31      46

AB != BA
Hence, this example demonstrates that matrix multiplication is not commutative.
```